

MEMO- APFLOW : 基于手机系统状态多模态数据和通用大模型推理的多用户多场景移动端应用推荐系统

项目概况

以安卓为代表的手机常有高频应用推荐,这种推荐往往是基于过去较长一段时间内的用户使用频次,推荐内容在短期内为静态,不能很好的反映用户需要。如果基于应用的历史使用序列,利用高阶马尔可夫,Transformer 或循环神经网络推荐,时间粒度会更小,但是用户的行为不是对历史数据的简单重复,同时这类方法会学习到噪声且泛化能力差,也不能有效地提升推荐系统的用户体验。这里,我们基于手机的系统状态信息设计了一个应用推荐系统,并在 Pixel5 移动端做了初步的测试,希望能实现移动端的高质量应用推荐。

在设计推荐算法时首要考虑的信息是用户意图和系统状态。用户意图不确定性大,难以系统地捕捉和准确预测,使用统计模型捕捉长时间周期下的高频度应用是最简单有效的方法。然而,系统状态的信息较稳定,可以用系统 UI 和 eBPF 等工具全面而高效地捕捉,系统状态的变化相对于用户行为乃至意图也更适合作为研究和预测的对象。对于推荐系统,考虑手机的系统状态主要有两个好处。第一是基于内存,中央处理单元,网络收发,电量等状态推荐应用,可以引导用户基于当前的系统状态选择应用,提升用户的使用体验;第二是单台手机上的高频应用数量不多,结合其使用的系统资源

类型对这些高频应用分类，可以准确聚焦到要推荐的三个应用。我们认为，基于长时间周期下的应用使用频次，以及短时间窗口下的系统状态，可以设计出高质量的移动端应用推荐系统，提升用户的手机使用体验。

基于系统状态信息，我们使用端侧通用大模型生成推荐结果。这里不使用一般调度算法，而使用大模型，主要有以下考量。第一，系统状态由内存，网络，电量，文件读写，乃至摄像头的使用等多个变量共同定义，属于多模态数据，一般的调度算法只能分析单一变量的使用情况，而端侧大模型可以基于语义综合考量系统状态，给出应用推荐。第二，应用推荐是一个多场景任务，系统在低负载，高负载，不同电量，不同网络环境等不同场景下会给出不同的应用推荐，通用大模型可适应多样化的场景。第三，我们可以让用户自愿填写个人信息，并基于用户画像生成专用提示词，让端侧大模型实现多用户场景下用户意图的个性化识别。不可否认的是，大模型的内存与运算资源开销使得它的移动端运用限制很大，但是推荐系统并不需要高频度的数据更新。使用高度压缩的格式化输入，每半小时至一小时进行 30-60 秒的推理，不会给系统造成太大的负担，也足以满足推荐系统的需要。

我们还编写了一个名为“MEMO- APPFLOW”的应用程序，作为该项目的前端。该应用的图标是一个橙色的“M”，图标背景为深蓝色。通过打开 MEMO 应用，用户就可以启动或停止该推荐系统。与此同时，MEMO 会在手机的进入页生成一个矩形

框显示被推荐的应用。通过点击矩形框中的应用图标，用户就可以打开对应的应用。除此之外，用户还可以在 MEMO 应用程序内查看对应的系统状态报告，编辑用户信息，调整运行模式。

我们的推荐系统 MEMO- APPFLOW 使用 eBPF 采集反映手机系统状态的多模态数据，利用端侧通用大模型推理，实现了多用户多场景下的移动端应用推荐。不同于很多方法对历史应用序列的过拟合，MEMO- APPFLOW 在长周期应用程序使用频率的基础上，根据系统状态的变化推荐，主动地向用户推荐应用程序，显示在手机首页的推荐栏上，能协助用户更便捷地实现手机应用切换，高效地利用系统资源。

程序实现

MEMO- APPFLOW 的核心程序包括两个部分：系统状态收集模块和大模型推理模块。这里将对这两个模块分别做具体的介绍。

系统状态收集模块的核心工具是 eBPF.在手机上使用 eBPF 需要管理员权限，在本项目中该功能由 magisk 实现；如果 MEMO- APPFLOW 将来能被更广泛的应用，则可由各手机厂商内置于系统中。eBPF 内置于安卓内核，所以本方法尤其适配于安卓生态。

这里我们只计算eBPF挂点被触发的次数，不考虑eBPF返回的系统数据。这么做主要有两个考量，如下所述。第一是eBPF挂点被触发，从最本质上反映了系统与硬件的状态。eBPF返回的PID，TID，网络端口等信息可以用来追踪系统中的一个具体进程，但不能反映系统的整体状态。第二是为了大模型输入的简洁性考虑。端侧大模型的吞吐量有限，只能接受几十个词元的输入，不能接受数百万条的系统记录。以下是一个大模型的部分输入示例：

```
"compressed_ebpf": [  
  {  
    "event_type": "MEMO_SCHED",  
    "detail": "sched_switch_or_wakeup",  
    "count": 281511,  
    "rate_per_sec": 70377.75  
  },  
  {  
    "event_type": "MEMO_RECVFROM",  
    "detail": "recvfrom",  
    "count": 56796,  
    "rate_per_sec": 14199  
  },  
  {  
    "event_type": "MEMO_SENDTO",  
    "detail": "sendto",  
    "count": 26825,  
    "rate_per_sec": 6706.25  
  },  
  {  
    "event_type": "MEMO_BINDER",  
    "detail": "binder_transaction",  
    "count": 16234,  
  }  
]
```

```
    "rate_per_sec": 4058.5
  },
  {
    "event_type": "MEMO_OPENAT",
    "detail": "openat",
    "count": 12420,
    "rate_per_sec": 3105
  },
  {
    "event_type": "MEMO_PROCESS_EXIT",
    "detail": "process_exit",
    "count": 350,
    "rate_per_sec": 87.5
  },
  {
    "event_type": "MEMO_PROCESS_FORK",
    "detail": "process_fork",
    "count": 347,
    "rate_per_sec": 86.75
  },
  {
    "event_type": "MEMO_PROCESS_EXEC",
    "detail": "process_exec",
    "count": 110,
    "rate_per_sec": 27.5
  },
  {
    "event_type": "MEMO_MEMORY",
    "detail": "reclaim_or_kswapd",
    "count": 16,
    "rate_per_sec": 4
  }
]
```

这里对不同的系统事件做了统计，我们可以看到系统调度的数量是最多的，其次是网络收发，然后才是数据交换和打开文件，之后是和进程相关的系统事件，最后是内存的交换与回收。在不完全统计下，一台手机每秒有近十万次的系统事件。对挂点被触发次数的统计本身也是很有意义的研究内容。

大模型推理使用的是 Qwen-0.8B-Q4_K_M 模型。推理的提示词如下：

```
"You are a mobile app usage predictor. Based on the predicted app category and additional user context, choose exactly one specific app ID.\n"
```

```
"You MUST output exactly one sentence in this format: This user will use App <id>.\n"
```

```
"The <id> MUST be one numeric ID from the candidate list below. Do not output a category name by itself.\n"
```

```
"Return only that sentence. Do not include reasoning, markdown, XML tags, or <think> blocks.\n"
```

```
"Example format: This user will use App 4.\n\n"
```

在接受系统状态输入后，端测大模型会生成结构化的输出，向用户推荐应用程序：

```
"recommendations": [  
  {  
    "package_name": "com.android.chrome",  
    "label": "Chrome",  
    "category": "Network IO",
```

```
    "confidence": 0.98
  },
  {
    "package_name": "com.google.android.apps.messaging",
    "label": "Messages",
    "category": "Communication",
    "confidence": 0.98
  },
  {
    "package_name": "com.google.android.GoogleCamera",
    "label": "Camera",
    "category": "Camera Service",
    "confidence": 0.9
  }
],
```

在这个例子中，由于网络收发频繁，系统向我们推荐了 CHROME 浏览器。

一般情况下，Qwen-0.8B-Q4_K_M 模型需要使用约 30 秒的推理时间，在推理期间会占用较多的内存和运算资源。

应用测试

在手机上，我们采用 3 分钟的时间窗口，采集到了以下的典型场景：

如果大量使用 QQ 收发文件，MEMO- APFLOW 会推荐 Chrome 和 Message. 这是由于与网络和通信相关的系统事件被大量触发，这些系统状态和资源对应的典型应用就会被推荐。

如果在三分钟的时间窗口内大量在不同应用间切换，与调度，进程相关的事件会被大量触发，与系统调度相关的典型应用 Settings 就会被推荐。

如果在三分钟的时间窗口内使用腾讯会议，与摄像头有关的事件就会被大量触发，与之相关的典型应用 Camera 就会被推荐。

从这些典型场景可以看出，即使 MEMO 的推荐结果还有待提升，但是它具备了捕捉用户行为背后的系统状态，并据此简单推理作出推荐的能力。如果能把这些系统状态与被推荐的应用程序做更好的关联，例如把这些典型应用换成用户高频使用的软件，或者推荐逻辑改为“使用不同的系统资源（对应的应用），并在不同系统资源间建立关联（如：网络被大量使用，就推荐与摄像头有关的应用）”，该推荐系统会有更好的表现。

除此之外，MEMO- APPFLOW 的项目文件中有一些关于准确率与性能提升的试验结果也可供参考。

总结与展望

由于运算开销大，实现以大语言模型为核心的人工智能操作系统可能难以实现。用户行为受内部、外部多种不确定因素影响，预测也比较困难。本项目基于 eBPF 和端侧大模型完成

的工作，更适用于应用推荐系统。根据方法和测试结果，本项目有以下的亮点：

第一，认识到了“eBPF 挂点被触发”在系统和硬件层面的意义，并以此为核心全面地监测系统的整体状态。

第二，测试了手机移动端能支持的语言模型的推理能力，运行效率和运行开销。

第三，对“基于系统状态做应用推荐”这一方案做了初步的测试。

如果要深入开展这个工作，下一步应该研究针对不同用户行为的典型场景，系统资源之间的依赖关系，并据此做应用推荐。此外，推荐算法可以将模型推理与调度算法相结合，尽可能减少模型推理的工作量。